Module 4: CREATE TABLE Statement

> The `CREATE TABLE` statement is one of the most fundamental SQL commands. Every database application begins with designing and creating tables that store data efficiently and accurately.

📖 Introduction

A table is the primary object used to store data in a relational database.

Before inserting, updating, or analyzing data, you must first create a table structure.

The `CREATE TABLE` statement allows you to:

- Define table names
- Define columns
- Assign data types
- Apply constraints
- Enforce data integrity

🎯 Learning Objectives

After completing this module, you will be able to:

✅ Create tables from scratch

✅ Define columns correctly

✅ Choose appropriate data types

✅ Apply constraints

✅ Follow naming conventions

✅ Design production-ready tables

✅ Understand table creation best practices

📚 Table of Contents

1️⃣ What is a Table?

A table stores data in rows and columns.

Example:

Employees Table

EmployeeID	EmployeeName	Salary
101	Kunal	45000
102	Rahul	50000
103	Aman	55000

Structure

```
Employees
│
├── EmployeeID
├── EmployeeName
└── Salary
```

Each column stores a specific type of information.

Each row represents one record.

2️⃣ What is CREATE TABLE?

`CREATE TABLE` is a DDL (Data Definition Language) command.

Used to:

- Create new tables
- Define columns
- Specify data types
- Apply constraints

3️⃣ CREATE TABLE Syntax

Basic syntax:

```
CREATE TABLE TableName
(
    Column1 DataType,
    Column2 DataType,
    Column3 DataType
);
```

Example

```
CREATE TABLE Students
(
    StudentID INT,
    StudentName VARCHAR(100),
    Age INT
);
```

Breakdown

Element	Description
CREATE TABLE	Creates a new table
Students	Table Name
StudentID	Column
INT	Data Type
VARCHAR(100)	Variable-length text

4️⃣ Creating Your First Table

Example:

```
CREATE TABLE Employees
(
    EmployeeID INT,
    EmployeeName VARCHAR(100),
    Salary DECIMAL(10,2),
    JoiningDate DATE
);
```

Table Structure

Column	Data Type
EmployeeID	INT
EmployeeName	VARCHAR(100)
Salary	DECIMAL(10,2)
JoiningDate	DATE

---

5 Column Definitions

Every column requires:

Column Name

Example:

```
EmployeeName
```

Data Type

Example:

```
VARCHAR(100)
```

Optional Constraint

Example:

```
EmployeeName VARCHAR(100) NOT NULL
```

Example

```
CREATE TABLE Products
(
    ProductID INT,
    ProductName VARCHAR(100),
    Price DECIMAL(10,2)
);
```

---

6 Using Data Types

Selecting correct data types is critical.

Employee Table

```
CREATE TABLE Employees
(
    EmployeeID INT,
    EmployeeName VARCHAR(100),
    Salary DECIMAL(10,2),
    JoiningDate DATE,
    IsActive BIT
);
```

Why These Types?

Column	Data Type	Reason
EmployeeID	INT	Numeric ID
EmployeeName	VARCHAR	Text
Salary	DECIMAL	Currency
JoiningDate	DATE	Date
IsActive	BIT	True/False

---

7 Adding Constraints

Constraints enforce rules.

PRIMARY KEY

Uniquely identifies records.

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(100)
);
```

NOT NULL

Requires value.

```
CREATE TABLE Employees
(
    EmployeeID INT,
    EmployeeName VARCHAR(100) NOT NULL
);
```

UNIQUE

Prevents duplicates.

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE
);
```

DEFAULT

Provides default value.

```
CREATE TABLE Employees
(
    Country VARCHAR(50)
    DEFAULT 'India'
);
```

CHECK

Validates data.

```
CREATE TABLE Employees
(
    Age INT CHECK (Age >= 18)
);
```

Complete Example

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,

    EmployeeName VARCHAR(100) NOT NULL,

    Email VARCHAR(100) UNIQUE,

    Salary DECIMAL(10,2)
    CHECK (Salary > 0),

    Country VARCHAR(50)
    DEFAULT 'India'
);
```

---

8 Creating Multiple Tables

Departments Table

```
CREATE TABLE Departments
(
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50)
);
```

Employees Table

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,

    EmployeeName VARCHAR(100),

    DepartmentID INT
);
```

Adding Foreign Key

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,

    EmployeeName VARCHAR(100),

    DepartmentID INT,

    FOREIGN KEY (DepartmentID)
    REFERENCES Departments(DepartmentID)
);
```

Relationship Diagram

```
Departments
│
├── DepartmentID (PK)
│
▼

Employees
```

```
│
├── EmployeeID (PK)
└── DepartmentID (FK)
```

9️⃣ Table Design Principles

One Table = One Entity

Good:

```
Employees
Departments
Products
```

Bad:

```
EmployeesDepartmentsProducts
```

Avoid Duplicate Columns

Bad:

```
Phone1
Phone2
Phone3
Phone4
```

Better:

Separate phone table.

Choose Proper Data Types

Good:

```
Salary DECIMAL(10,2)
```

Bad:

```
Salary VARCHAR(100)
```

🔟 Naming Conventions

Consistent naming improves readability.

Table Names

Good:

```
Employees
Products
Orders
```

Avoid:

```
tblEmp
EMP
e1
```

Column Names

Good:

```
EmployeeID
EmployeeName
JoiningDate
```

Avoid:

```
EmpID
Nm
JD
```

Primary Keys

Convention:

```
TableNameID
```

Examples:

```
EmployeeID
ProductID
OrderID
```

Foreign Keys

Same name as referenced key.

Example:

```
DepartmentID
```

1️⃣1️⃣ Real-World Examples

Example 1: Student Table

```sql
CREATE TABLE Students
(
    StudentID INT PRIMARY KEY,

    StudentName VARCHAR(100) NOT NULL,

    Age INT CHECK(Age >= 16),

    Email VARCHAR(100) UNIQUE
);
```

Example 2: Product Table

```
CREATE TABLE Products
(
    ProductID INT PRIMARY KEY,

    ProductName VARCHAR(100) NOT NULL,

    Price DECIMAL(10,2)
    CHECK(Price > 0),

    StockQuantity INT DEFAULT 0
);
```

Example 3: Orders Table

```
CREATE TABLE Orders
(
    OrderID INT PRIMARY KEY,

    CustomerID INT,

    OrderDate DATE,

    TotalAmount DECIMAL(10,2)
);
```

1️⃣ 2️⃣ Common Mistakes

Using Wrong Data Types

❌

```
Salary VARCHAR(100)
```

✅

```
Salary DECIMAL(10,2)
```

Missing Primary Keys

Bad:

```
CREATE TABLE Employees
(
    Name VARCHAR(100)
);
```

Good:

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100)
);
```

Allowing NULL Everywhere

Bad:

```
Name VARCHAR(100)
```

Better:

```
Name VARCHAR(100) NOT NULL
```

Poor Naming

Bad:

```
CREATE TABLE T1
(
    C1 INT
);
```

Good:

```sql
CREATE TABLE Employees
(
    EmployeeID INT
);
```

1️⃣3️⃣ Best Practices

Always Define Primary Keys

Every table should have one.

Use Meaningful Names

Avoid abbreviations.

Apply Constraints

Protect data quality.

Use Appropriate Data Types

Optimize storage.

Keep Table Structure Simple

One table should represent one entity.

Plan Relationships Early

Identify foreign keys during design.

Example: Production-Ready Table

```sql
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,

    EmployeeName NVARCHAR(100) NOT NULL,

    Email VARCHAR(150) UNIQUE,
```

```
    Salary DECIMAL(10,2)
    CHECK(Salary > 0),

    JoiningDate DATE NOT NULL,

    IsActive BIT DEFAULT 1,

    CreatedAt DATETIME2 DEFAULT GETDATE()
);
```

📝 Summary

In this module, you learned:

✅ What a table is

✅ CREATE TABLE syntax

✅ Column definitions

✅ Data types

✅ Constraints

✅ Primary Keys

✅ Foreign Keys

✅ Table relationships

✅ Naming conventions

✅ Database design principles

✅ Production-ready table design

🧠 Practice Questions

Theory

1. What is CREATE TABLE?
2. Why are data types important?
3. What is a primary key?
4. What is a foreign key?
5. Why use constraints?
6. What is NOT NULL?
7. What is UNIQUE?

8. What is DEFAULT?
9. What is CHECK?
10. What are naming conventions?

Practical Exercises

Task 1

Create a table:

```
Students
```

Columns:

```
StudentID
StudentName
Age
Email
```

Requirements:

- StudentID = Primary Key
- Age >= 16
- Email Unique

Task 2

Create:

```
Departments
Employees
```

Add:

- Primary Key
- Foreign Key

Task 3

Create Product Table

Columns:

```
ProductID
ProductName
Price
StockQuantity
CreatedDate
```

Apply appropriate constraints.

Challenge Exercise

Design a database for:

```
Library Management System
```

Create tables:

- Books
- Members
- BorrowRecords

Add:

- Primary Keys
- Foreign Keys
- Constraints

🚀 Next Module

➞ Module 5: Constraints

Topics Covered:

- NOT NULL
- UNIQUE
- PRIMARY KEY
- FOREIGN KEY
- CHECK
- DEFAULT
- Constraint Best Practices
- Real-World Data Validation